

Progetto:

WattGuard

Titolo del documento:

Analisi e Progettazione

Document Info

Doc. Name	D3-WattGuard-DescrizioneProgetto	Doc. Number	D3 V1.0
Description	Documento di analisi e progettazione		

Authors

Alessio Pesaresi	251783
Michele Porcellato	254581
Marco Piovesan	255380

Indice

1	Analisi dei componenti	2
1.1	Definizione dei componenti	2
1.1.1	Gestione autenticazione	2
1.1.2	Gestione database	3
1.1.3	Acquisizione dati real-time	4
1.1.4	Integrazione meteo	4
1.1.5	Gestione dashboard e grafici	4
1.1.6	Gestione edifici e sensori	5
1.1.7	Sistema di allerta e notifiche	6
1.1.8	Gestione esportazione	6
1.1.9	Gestione utenti	7
1.1.10	Motore di ricerca edifici	7
1.1.11	Gestione autorizzazioni	8
1.1.12	Analisi comparativa	8
1.1.13	Servizio comunicazioni (email)	9
1.2	Diagramma dei componenti	10
2	Diagramma delle classi	11
2.1	Definizione delle classi	11
2.2	Diagramma delle classi complessivo	17
3	Dal Class Diagram alle API	18

1 Analisi dei componenti

Nel presente capitolo viene presentata l'architettura in termini di componenti interni al sistema **WattGuard**, definiti sulla base dei requisiti analizzati in precedenza. L'architettura adotta un modello a dati centralizzato: tutti i dati (provenienti dai sensori e dalle API meteo) vengono prima acquisiti e resi persistenti nel database, per poi essere messi a disposizione di tutti gli altri componenti che li richiedono esclusivamente attraverso il database stesso.

Nota di conformità all'implementazione. I nomi di campi e riferimenti usati nel documento corrispondono ai nomi reali dei modelli Mongoose implementati: `building` (non `buildingId`), `sensor` (non `sensorId`), `metadata.sensor` e `metadata.building` (non `metadata.sensorId` / `metadata.buildingId`). Nelle risposte JSON delle API l'identificativo è esposto come `_id` (con `self` come link canonico) e i riferimenti sono risolti via `populate`.

1.1 Definizione dei componenti

Definiamo di seguito i componenti del sistema con le relative funzionalità.

1.1.1 Gestione autenticazione

Descrizione

Il componente si occupa della funzionalità di autenticazione degli utenti (Amministratore e Operatore Tecnico) tramite credenziali locali (email/password con hashing) o tramite autenticazione federata con Google (SSO). Include le funzionalità di login e reset della password. Il logout è esclusivamente client-side (rimozione del JWT da `localStorage`, nessun endpoint di revoca server-side perché i token sono stateless con scadenza di 8 ore). L'onboarding dei nuovi operatori avviene tramite invite-link e non tramite credenziali temporanee generate.

Interfaccia richiesta - Credenziali locali:

Riceve dall'utente l'inserimento di email e password per l'accesso al sistema. Le password sono memorizzate utilizzando algoritmi di hashing robusti (bcrypt, costo 10) con salt individuale.

Interfaccia richiesta - Autenticazione Google:

Riceve l'autorizzazione all'accesso al sistema proveniente dall'SSO di Google, previo controllo delle credenziali inserite dall'utente nei sistemi Google. L'account Google deve essere pre-registrato tramite invito: in caso contrario l'accesso è rifiutato (`oauth.no.account`). Il login locale e quello Google condividono lo stesso endpoint `POST /auth/session`.

Interfaccia fornita - Richiesta a Google:

Provvede alla configurazione esposta da `GET /auth/google/config`; la verifica del token `idToken` avviene server-side con `google-auth-library` (nessun redirect URI/secret dedicato).

Interfaccia fornita - Autenticazione e ruolo:

Fornisce il token JWT (`Authorization: Bearer`, scadenza 8 ore) e il ruolo dell'utente (Amministratore o Operatore), permettendo l'accesso alle funzionalità corrispondenti.

Interfaccia fornita - Attivazione credenziali:

L'amministratore crea un invito (POST /invites, scadenza 7 giorni) e il nuovo utente lo accetta tramite PATCH /invites/:id impostando la password locale oppure collegando l'account Google; non vengono generate né inviate password in chiaro.

1.1.2 Gestione database**Descrizione**

Punto centrale di archiviazione e unico punto di accesso ai dati (unica connessione Mongoose verso MongoDB). Gestisce i dati storici, le configurazioni di edifici e sensori, le credenziali locali, i log di sistema e i report. Non è implementato alcun backup automatico giornaliero con retention di 30 giorni: la retention configurabile (`database.dataRetentionDays`, default 365 giorni) si applica alle rilevazioni tramite TTL sulla collezione time-series `sensorreadings`. Tutti i componenti del sistema richiedono i dati esclusivamente attraverso questo componente.

Interfaccia richiesta - Dati sensori:

Riceve le rilevazioni dei sensori per l'archiviazione storica.

Interfaccia richiesta - Dati meteo:

Riceve i dati della temperatura esterna per l'archiviazione e il successivo utilizzo.

Interfaccia richiesta - Dati modificati:

Riceve le modifiche apportate ai dati per la persistenza.

Interfaccia fornita - Dati storici:

Fornisce le rilevazioni archiviate per l'intervallo temporale selezionato.

Interfaccia fornita - Dati in tempo reale:

Fornisce le ultime rilevazioni disponibili per il monitoraggio live.

Interfaccia fornita - Temperatura esterna:

Fornisce i dati meteo archiviati per l'analisi comparativa dell'efficienza termica.

Interfaccia fornita - Metadati edificio:

Fornisce i dettagli strutturali degli edifici (nome, indirizzo, superficie, tipologia impianto) e la configurazione dei sensori associati.

Interfaccia fornita - Soglie personalizzate:

Fornisce i limiti configurati per temperatura e consumo.

Interfaccia fornita - Validazione email:

Verifica l'unicità dell'indirizzo email nel database.

Interfaccia fornita - Dati per backup:

Fornisce l'intero set di dati per l'esportazione on-demand tramite `GET /backups` (riservato all'amministratore). Non include la collezione `sensorreadings`, non prevede scheduling automatico, storage dedicato né ripristino.

1.1.3 Acquisizione dati real-time

Descrizione

Gestisce la ricezione delle misurazioni dai termometri smart e dai contatori connessi con una frequenza media di 90 secondi (`transmissionInterval`, default 90, configurabile tra 10 e 3600 secondi). Riceve i pacchetti dati dai dispositivi IoT e li rende disponibili esclusivamente per l'archiviazione nel database. L'ingestione MQTT (`sensors+/readings`) è disabilitata di default (`MQTT_ENABLED=false`) e la sorgente effettivamente attiva è il simulatore in-process, che emette con la stessa cadenza e invoca lo stesso servizio di ingestione (scrittura time-series, deduplica soglie in transazione, denormalizzazione di `lastReading`).

Interfaccia richiesta - Connessione sensori:

Riceve i pacchetti dati dai dispositivi IoT (termometri e contatori) tramite protocollo MQTT.

Interfaccia fornita - Dati sensori:

Invia i dati correnti per l'archiviazione e la successiva distribuzione agli altri componenti.

1.1.4 Integrazione meteo

Descrizione

Si occupa di reperire i dati della temperatura ambientale esterna tramite API meteo esterne (Open-Meteo, con geocoding via Nominatim OpenStreetMap) senza scheduling orario dedicato: i dati esterni sono usati come fallback per i calcoli di efficienza quando manca un sensore `external_temp`. Il frontend interroga direttamente `api.open-meteo.com` (coordinate di Trento hardcoded, `staleTime` 5 minuti) per l'interazione comparativa.

Interfaccia richiesta - API esterne:

Richiede dati ai provider meteo tramite chiamate REST (backend: archivio orario Open-Meteo; frontend: forecast corrente Open-Meteo).

Interfaccia fornita - Dati meteo:

Invia i dati climatici per l'archiviazione e la successiva distribuzione agli altri componenti.

1.1.5 Gestione dashboard e grafici

Descrizione

Genera la panoramica dei consumi e i grafici storici aggregati. Elabora i dati grezzi per visualizzare indici di performance e andamenti temporali (orari, giornalieri e settimanali su `GET /metrics/timeseries`; giornalieri nei report; mensili e annuali non implementati). Nessuno SLO di caricamento entro 60 secondi per il 95% delle richieste è misurato nell'implementazione.

Interfaccia richiesta - Dati storici:

Riceve le rilevazioni archiviate per l'intervallo selezionato.

Interfaccia richiesta - Dati in tempo reale:

Riceve le ultime rilevazioni disponibili con polling guidato da `settings.polling.intervalSeconds` (default 90 secondi; fallback frontend 120 secondi per le statistiche dashboard, 30 secondi di `staleTime` per il dettaglio edificio).

Interfaccia richiesta - Temperatura esterna:

Riceve i dati meteo archiviati per i calcoli di efficienza.

Interfaccia fornita - Visualizzazione:

Fornisce al front-end i grafici interattivi e gli indicatori disponibili in `GET /metrics` (sensori attivi/totali, allarmi attivi, consumi elettrici e gas come somma delle ultime letture; le temperature medie sono disponibili solo nel drill-down di edificio, non nei KPI aggregati).

Interfaccia fornita - Indicatori performance:

Fornisce i calcoli di efficienza (`GET /buildings/:id/efficiency`) per il confronto tra edifici.

1.1.6 Gestione edifici e sensori**Descrizione**

Gestisce l'anagrafica degli edifici pubblici (nome, indirizzo, superficie, anno costruzione, tipologia impianto) e la configurazione dei sensori associati (termometri smart e contatori). Permette di aggiungere, modificare e rimuovere edifici e sensori. Ogni modifica viene registrata con timestamp e identificativo dell'operatore.

Interfaccia richiesta - Autorizzazione:

Riceve il ruolo dell'utente (`admin` o `operator`, trasportato nel claim JWT e verificato dai middleware `requireOperator/requireAdmin`) che segnala se l'utente ha il permesso necessario per compiere un'azione.

Interfaccia richiesta - Metadati edificio:

Riceve i dettagli strutturali e la configurazione dei sensori per le operazioni di visualizzazione e modifica.

Interfaccia fornita - Dati edificio aggiornati:

Invia le modifiche apportate agli edifici e ai sensori per la persistenza, con audit tramite `createdBy/updatedBy` e `createdAt/updatedAt` (nessun log versionato immutabile separato).

Interfaccia fornita - Evento modifica:

Notifica ogni operazione di modifica, aggiunta o rimozione di edifici e sensori.

1.1.7 Sistema di allerta e notifiche

Descrizione

Monitora costantemente le rilevazioni rispetto alle soglie configurate sul sensore (`minThreshold/maxThreshold`) e alla soglia di efficienza (`efficiencyThresholds.minCop`, verificata da un job ogni 30 minuti su finestra 24h) e genera notifiche automatiche in caso di anomalie o malfunzionamenti. Le notifiche vengono visualizzate in un pannello dedicato con tab `active/acknowledged/resolved` e ciclo di vita `active → acknowledged → resolved` tramite `PATCH /alerts/:id`; possono essere prese in carico e chiuse dall'operatore dopo la risoluzione del problema.

Interfaccia richiesta - Soglie personalizzate:

Riceve i limiti impostati (es. temperatura min/max, consumo massimo orario, variazioni anomale) dagli attributi di sensore ed edificio.

Interfaccia richiesta - Dati in tempo reale:

Riceve le ultime rilevazioni disponibili per la verifica in tempo reale delle soglie, con deduplica (un solo allarme `active` per sensore/tipo).

Interfaccia richiesta - Autorizzazione:

Riceve il ruolo dell'utente (`admin` o `operator`) che segnala se l'utente ha i permessi per visualizzare e gestire gli allarmi.

Interfaccia fornita - Allarmi:

Genera toast in-app ed email broadcast (nessuna notifica desktop/push nativa) oltre alle voci nel pannello dedicato per la gestione degli interventi, ordinate per `createdAt` discendente lato backend.

Interfaccia fornita - Evento allarme:

Notifica la generazione, la presa in carico e la chiusura di ogni allarme (con `acknowledgedBy/acknowledgedAt` e `resolvedBy/resolvedAt` come nome utente e data).

1.1.8 Gestione esportazione

Descrizione

Genera report riservati all'amministratore, per singoli edifici o gruppi di edifici e per periodi temporali specifici (`buildingIds`, `startDate`, `endDate`). I dati grezzi sono esportati da `GET /readings` in formato CSV e JSON; i report aggregati (riepilogo + giornaliero con grafico) sono esportati da `GET /reports` in formato PDF e XLSX. Include dati anagrafici, consumi, temperature, timestamp e anomalie a seconda della sorgente.

Interfaccia richiesta - Dati per export:

Riceve i dati storici filtrati per edificio e periodo temporale.

Interfaccia richiesta - Autorizzazione esportazione:

Riceve il ruolo dell'utente e verifica che sia Amministratore (`requireAdmin`) per esportare i dati.

Interfaccia fornita - Download:

Fornisce il trigger per il download immediato dei dati filtrati. Il CSV è formattato correttamente per l'importazione in Excel.

1.1.9 Gestione utenti**Descrizione**

Permette all'amministratore comunale di gestire gli account degli operatori tecnici. Si occupa della creazione (esclusivamente tramite inviti, nessuna `POST /users`), modifica (`role`, `isDisabled`), eliminazione e gestione dei permessi dei profili utente, con protezione anti auto-modifica. Il sistema invia automaticamente una email all'operatore con il link di invito e le istruzioni per il primo accesso (impostazione password o collegamento Google); non vengono inviate credenziali in chiaro.

Interfaccia richiesta - Validazione email:

Verifica l'unicità dell'indirizzo email inserito prima di procedere alla creazione dell'invito.

Interfaccia richiesta - Template messaggio:

Riceve il template dell'email da inviare al nuovo operatore.

Interfaccia fornita - Gestione account:

Fornisce le funzionalità per inserire email e permessi dell'utente.

Interfaccia fornita - Credenziali generate:

Fornisce l'invito con token (hash SHA-256, scadenza 7 giorni) accettabile dal nuovo operatore/amministratore tramite `PATCH /invites/:id`; se l'invio email fallisce, l'invito viene annullato.

Interfaccia fornita - Richiesta invio email:

Richiede l'invio effettivo dell'email di benvenuto con link `/accept-invite?token=`.

Interfaccia fornita - Evento gestione utenti:

Notifica ogni operazione di creazione, modifica o eliminazione di account.

1.1.10 Motore di ricerca edifici**Descrizione**

Implementa la logica di ricerca degli edifici pubblici monitorati. L'API (`GET /buildings`) permette di filtrare in combinazione AND per nome, indirizzo, zona geografica, tipologia di struttura (per esempio scuola, ospedale, ecc.) e stato, con paginazione, ordinamento e indici dedicati. Il componente di ricerca del frontend inoltra al server il solo filtro di stato e applica gli altri filtri (nome, indirizzo, tipologia) lato client; nessun campo di ricerca per zona è esposto nella UI. Nessuno SLO di completamento entro 10 secondi è misurato nell'implementazione.

Interfaccia richiesta - Dati anagrafici:

Riceve la lista degli edifici che corrispondono ai criteri inseriti.

Interfaccia fornita - Risultati ricerca:

Restituisce una lista di edifici completa di dettagli quali superficie, stato operativo e tipologia di impianto.

1.1.11 Gestione autorizzazioni**Descrizione**

Determina a quali funzionalità un utente può accedere in base al ruolo assegnato (Operatore o Amministratore). Controlla, ad esempio, se un utente ha il permesso di esportare dati o gestire altri utenti. Non è usata alcuna bitmask: il ruolo è un enum (`admin` | `operator`) trasportato nel JWT e verificato dai middleware `requireOperator` (`admin` + `operator`) e `requireAdmin` (solo `admin`).

Interfaccia richiesta - Ruolo utente:

Riceve l'identità certificata e il ruolo dell'utente loggato.

Interfaccia fornita - Bit di autorizzazione:

Fornisce l'esito dei controlli di ruolo che determinano se l'utente ha il permesso necessario per compiere un'azione.

1.1.12 Analisi comparativa**Descrizione**

Componente dedicato al confronto delle performance tra diversi edifici. Permette di selezionare al massimo 3 edifici (`MAX_COMPARE_BUILDINGS=3`, limite rigido) per confrontarne superficie, sensori attivi, consumo corrente, consumo per m², tipologia, anno, impianto, zona ed efficienza (COP, dispersione, kWh totali). Non sono implementati costi stimati, temperature medie nella vista comparativa né badge di edificio più/meno efficiente: i valori sono solo affiancati.

Interfaccia richiesta - Indicatori performance:

Riceve i calcoli di efficienza relativi agli edifici selezionati.

Interfaccia richiesta - Metadati edificio:

Riceve i dettagli anagrafici degli edifici da confrontare.

Interfaccia fornita - Confronto visivo:

Genera tabelle comparative e grafici affiancati per evidenziare gli edifici più e meno efficienti.

1.1.13 Servizio comunicazioni (email)

Descrizione

Gestisce l'invio automatico di comunicazioni verso l'esterno tramite il provider Resend via HTTP (non via server SMTP dedicato). È responsabile della spedizione delle email con il link di invito ai nuovi operatori, dei link per il reset della password e delle notifiche di allarme (creazione/risoluzione, in modalità fire-and-forget se `notifications.emailEnabled` è attivo).

Interfaccia richiesta - Richiesta invio:

Riceve la richiesta di invio email con i dettagli del destinatario e il contenuto.

Interfaccia fornita - Invio mail:

Si interfaccia con le API Resend per il recapito effettivo dei messaggi agli indirizzi email degli utenti.

Interfaccia fornita - Template messaggi:

Fornisce i template predefiniti per le email di benvenuto (con link di invito), di reset password e di notifica allarme.

1.2 Diagramma dei componenti

Viene riportato di seguito un diagramma complessivo di tutti i componenti di sistema e le loro interconnessioni descritte in precedenza. Come si evince dal diagramma, il componente “Gestione database” rappresenta il punto centrale di raccolta e distribuzione dei dati, garantendo che tutti gli altri componenti accedano ai dati esclusivamente attraverso di esso.

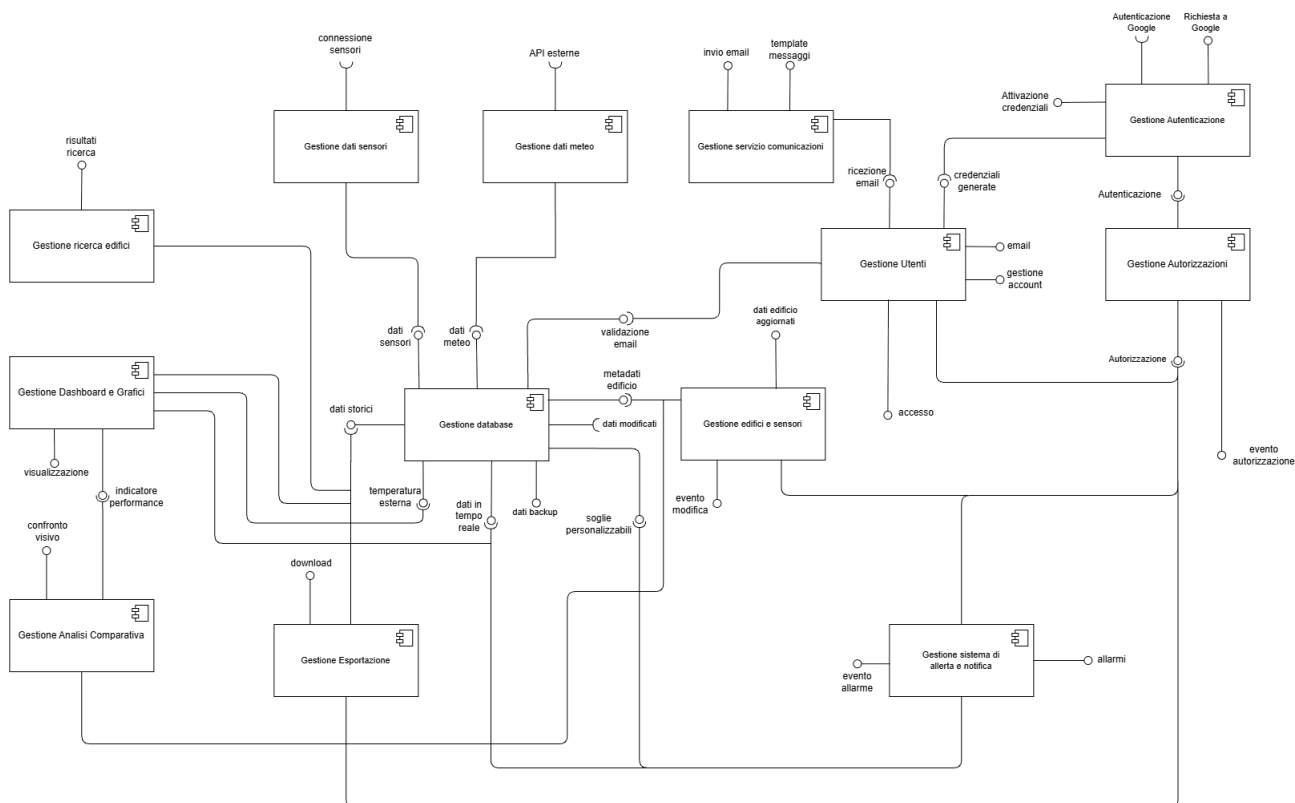


Figura 1: Diagramma dei componenti di WattGuard

2 Diagramma delle classi

2.1 Definizione delle classi

Le figure seguenti riportano i nomi reali dei campi Mongoose implementati. Nelle risposte JSON l'identificativo è esposto come `_id` (con `self` come link canonico). Ogni classe riporta oltre agli attributi anche i metodi (operazioni). La mappatura completa metodo → endpoint REST è nel capitolo 3.

1. Utente (User)

La classe `Utente` rappresenta qualsiasi utente del sistema. Il ruolo viene memorizzato nell'attributo `role` (enum `admin` o `operator`): l'operatore tecnico gestisce edifici, sensori e allarmi mentre l'amministratore possiede il controllo totale (gestione utenti, inviti, esportazione dati, configurazione di sistema oltre ai privilegi dell'operatore tecnico).

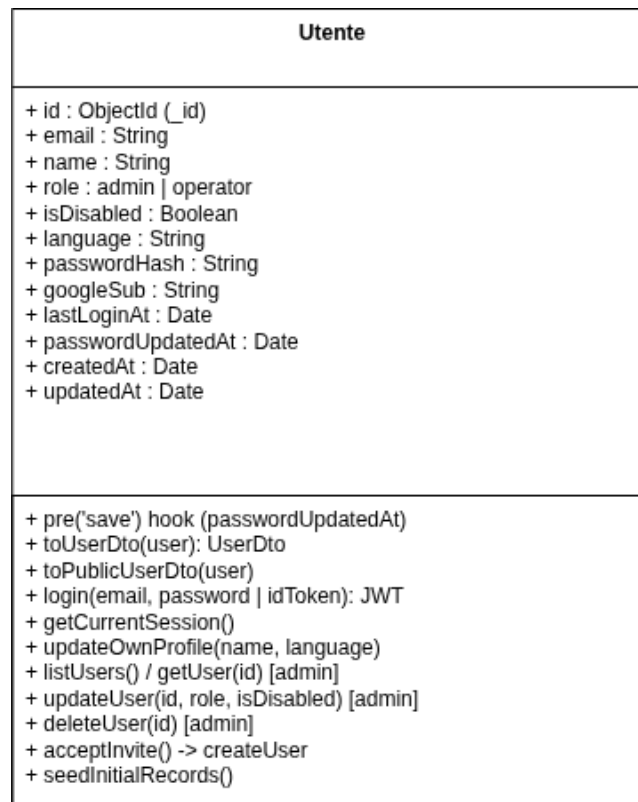


Figura 2: Classe `Utente`: attributi e metodi

2. Invito (Invite)

La classe `Invito` rappresenta l'invito inviato via email a un nuovo utente, con il relativo token e la scadenza (7 giorni). Permette all'amministratore di creare gli account senza condividerne le credenziali.

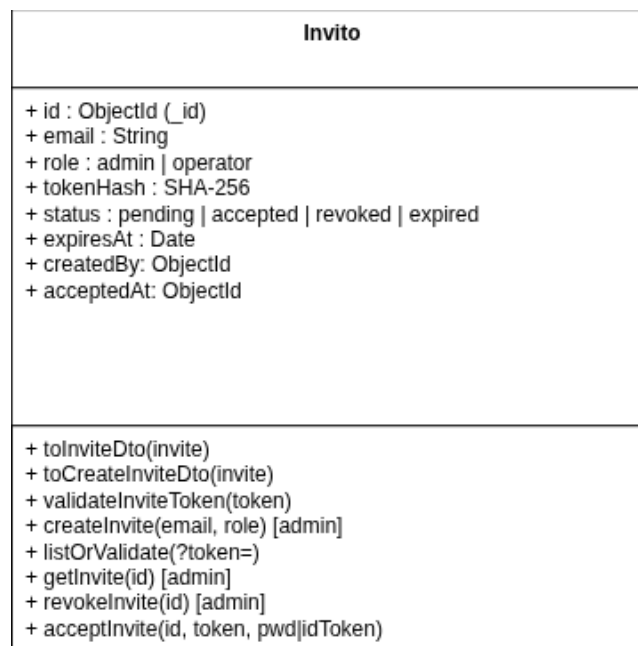


Figura 3: Classe Invito: attributi e metodi

3. TokenResetPassword (PasswordResetToken)

La classe `TokenResetPassword` rappresenta il token monouso generato per la procedura di reset della password. Il token scade dopo 1 ora (durata impostata dal service).

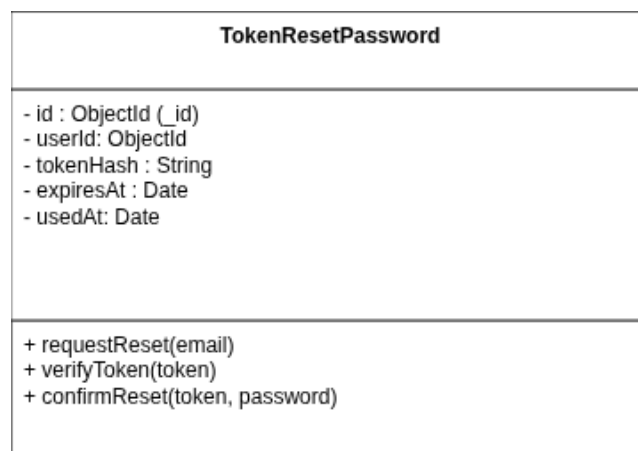


Figura 4: Classe TokenResetPassword: attributi e metodi

4. Edificio (Building)

La classe `Edificio` rappresenta la struttura pubblica monitorata dal sistema. Oltre ai dati anagrafici e strutturali, memorizza la posizione geografica come punto GeoJSON (longitudine/latitudine) e conserva il riferimento alla propria tipologia.

Edificio
+ id : ObjectId (_id) + name : String + address : String + surface : Number + ceilingHeight : Number = 3.0 + location : GeoJSON Point [lng, lat] + buildingType : ObjectId + heatingSystemType : String + constructionYear : Number + geographicZone : Number + status : String + efficiencyThresholds: Object {enabled, minCop} + createdBy : ObjectId + updatedBy! : ObjectId
+ toBuildingSummaryDTO() / DetailDTO() + searchBuildings(filtri) + createBuilding(dati) + getBuilding(id) + updateBuilding(id, dati) + deleteBuilding(id) cascade + getHistory(id) /readings + calculateBuildingEfficiency() + classifyHeatingSystem() + aggregateDailyConsumption() + aggregateConsumption() + buildReportData() + serialize Pdf/Xlsx + getCoordinates() / getAvgHistTemp() + evaluateEfficiencyAlerts() [cron] + advanceThermostat() / computeReading()

Figura 5: Classe Edificio: attributi e metodi

5. TipologiaEdificio (BuildingType)

La classe TipologiaEdificio rappresenta le categorie di edifici (ad esempio “scuola”, “ufficio”, “palestra”). Il nome è univoco; una tipologia può essere eliminata solo se non è più utilizzata da alcun edificio.

TipologiaEdificio
+ id : ObjectId (_id) + name : String + description: String + createdAt: String + updatedAt : Date
+ toBuildingTypeDTO(bt) + listBuildingTypes() + getBuildingType(id) + createBuildingType(dati) + updateBuildingType(id, dati) + deleteBuildingType(id)

Figura 6: Classe TipologiaEdificio: attributi e metodi

6. Sensore (Sensor)

La classe **Sensore** rappresenta un dispositivo IoT installato in un edificio. Il riferimento all'edificio è memorizzato nel campo **building** (nome reale nel codice). La tipologia è un attributo enumerato (temperatura interna, temperatura esterna, contatore elettrico, contatore gas); le soglie di allarme **minThreshold** e **maxThreshold** sono attributi del sensore stesso. L'ultima rilevazione è memorizzata in forma denormalizzata nell'attributo **lastReading** per garantire letture immediate, mentre lo storico completo è nella classe **Rilevazione**.

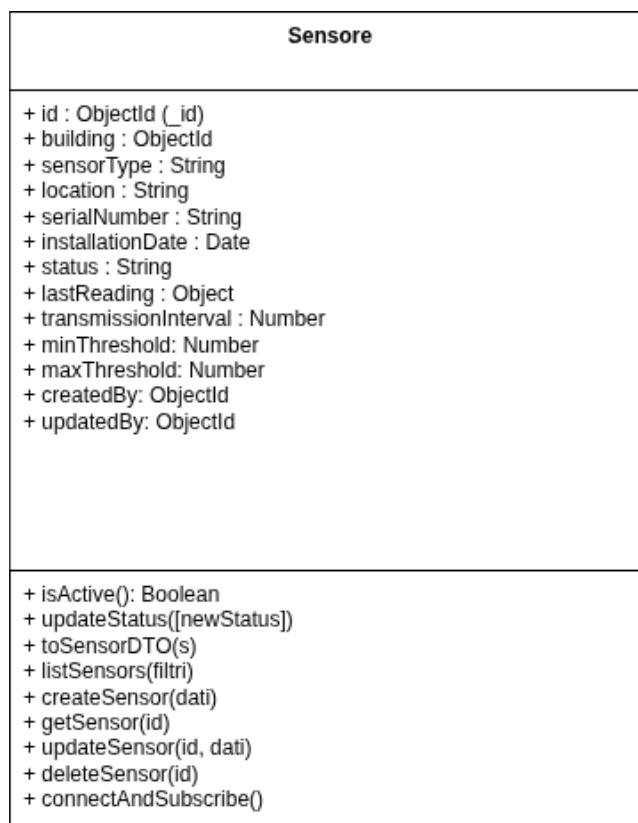


Figura 7: Classe **Sensore**: attributi e metodi

7. Rilevazione (SensorReading)

La classe **Rilevazione** rappresenta una singola misurazione effettuata da un sensore in un determinato istante. È una collezione che mantiene lo storico dei dati necessario alle analisi di andamento, efficienza e all'esportazione. Un TTL di 365 giorni rimuove automaticamente le rilevazioni più vecchie.

Rilevazione
+ id : ObjectId (_id) + timestamp : Date + value : Number + unit : String + metadata.sensor : ObjectId + metadata.building : ObjectId + metadata.sensorType : String
+ toSensorReadingDTO(r) + ingestReading(input) + handleMqttMessage(topic, msg) + computeSensorReading() + listBySensor(id) + listByBuilding(id) + exportReadings(format) + getTimeseries(hour day week) + serializeCsv()

Figura 8: Classe Rilevazione: attributi e metodi

8. Allarme (Alert)

La classe **Allarme** rappresenta una notifica generata automaticamente dal sistema quando una rilevazione supera le soglie configurate sul sensore oppure quando viene rilevata un'anomalia di efficienza (COP sotto soglia).

Allarme
+ id : ObjectId (_id) + building: ObjectId + sensor: ObjectId + type : String + thresholdType : min max + severity : low medium high critical + value: number + unit: String + limit: number + status : active acknowledged resolved + acknowledgedBy : ObjectId + acknowledgeAt: Date + resolvedBy: ObjectId + resolvedAt: Date
+ computeDeviationSeverity(value, limit) + raiseThreshold(dati) + raiseEfficiency(building, cop) + acknowledge(id, actor) + resolveManually(id, actor) + resolveEfficiencyForBuilding(b) + pruneForRemovedThresholds() + deleteForBuilding(b) / deleteForSensor(s) + toAlertDTO(a) + listAlerts(filtri) + getAlert(id) + dispatchAlertNotifications()

Figura 9: Classe Allarme: attributi e metodi

9. Configurazione Sistema (SystemConfig)

La classe `ConfigurazioneSistema` è un documento *singleton* solo per convenzione applicativa che conserva la configurazione globale del sistema (intervallo di polling del frontend, abilitazione delle notifiche email, giorni di conservazione dei dati storici).

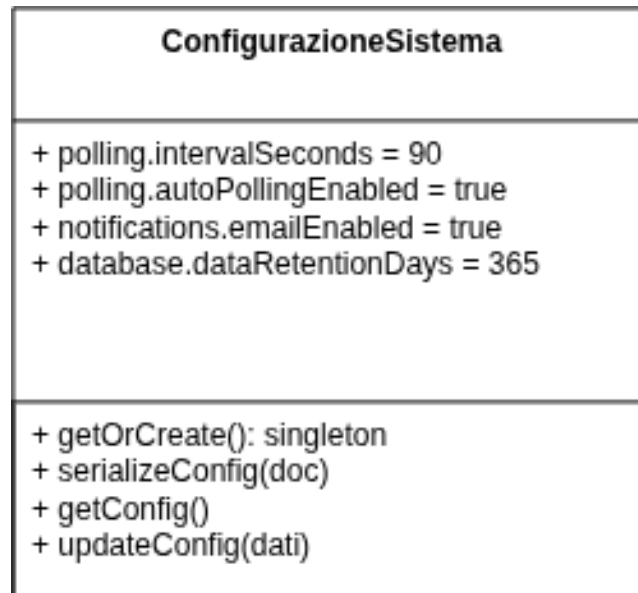


Figura 10: Classe `ConfigurazioneSistema`: attributi e metodi

2.2 Diagramma delle classi complessivo

Di seguito il diagramma delle classi complessivo con tutte le classi presentate e le relative relazioni (da rigenerare in base alle definizioni corrette).

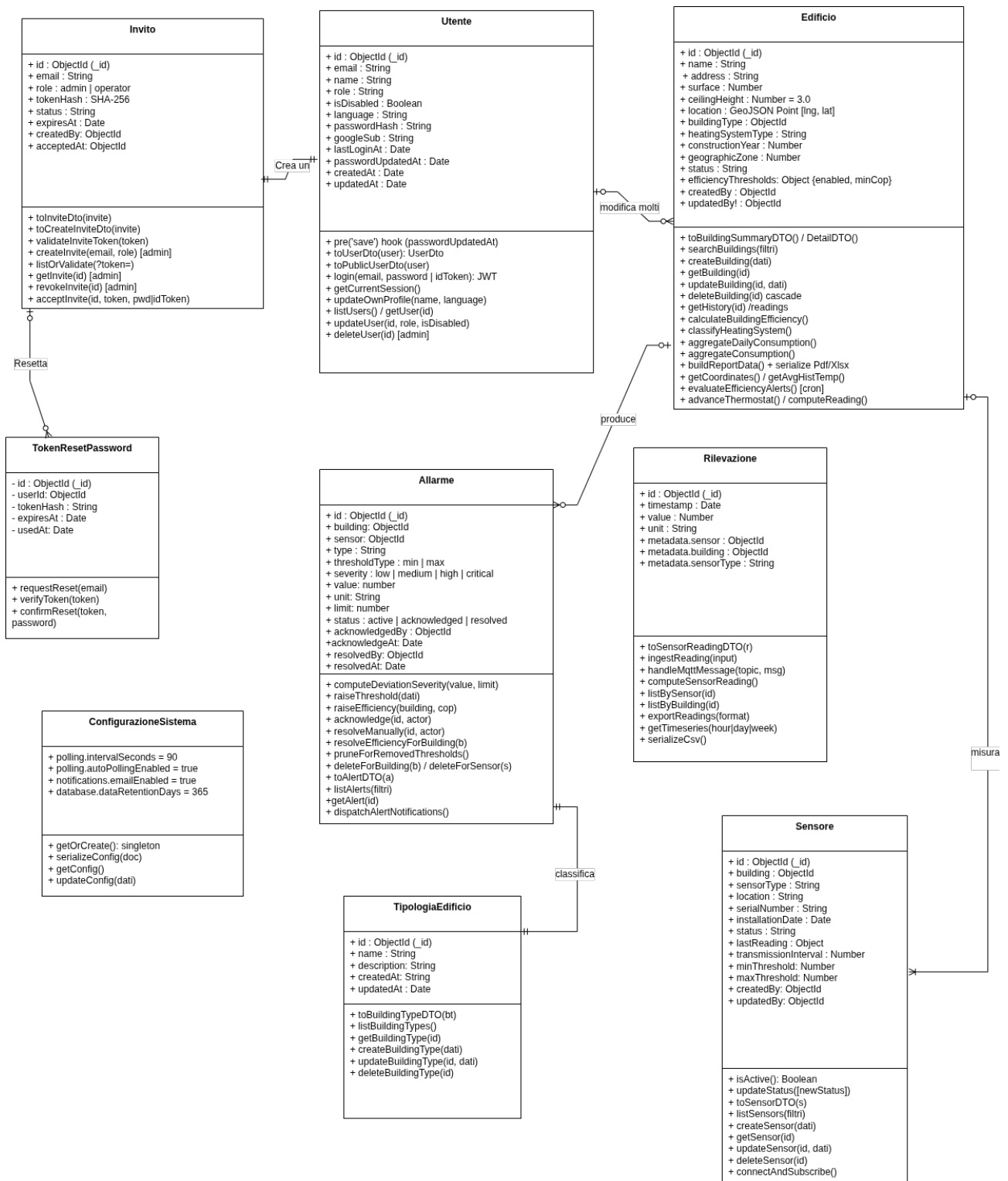


Figura 11: Diagramma delle classi complessivo di WattGuard

3 Dal Class Diagram alle API

Riportiamo di seguito una tabella con la mappatura tra i metodi presenti nel diagramma delle classi del capitolo precedente e le API individuate nel documento D3. L'uso del simbolo * indica che esso è un parametro di percorso. Nei parametri query invece il simbolo ! indica che il parametro è obbligatorio mentre ? è facoltativo.

Metodo	HTTP	URI + query	Request	Response
Classe Utente — autenticazione				
login()	POST	/auth/session	{email, password idToken}	{user, token}
getCurrentSession()	GET	/auth/session	-	User
updateOwnProfile()	PATCH	/auth/session	{name, language}	User
getGoogleClientId()	GET	/auth/google/config	-	{clientId}
logout()	-	-	-	-
Classe Utente — gestione account (solo admin)				
listUsers()	GET	/users/	-	User[]
getUser()	GET	/users/:id*	-	User
updateUser()	PATCH	/users/:id*	{role, isDisabled}	User
deleteUser()	DELETE	/users/:id*	-	204
acceptInvite()	PATCH	/invites/:id*	vedi Invito	{user, token}
toUserDto()	-	-	-	User
toPublicUserDto()	-	-	-	User
pre('save')	-	-	-	-
seedInitialRecords()	-	-	-	-
Classe Invito				
createInvite()	POST	/invites/	{email, role}	Invite
listOrValidate()	GET	/invites/ [?token=]	?token=	Invite Invite[]
getInvite()	GET	/invites/:id*	-	Invite
revokeInvite()	DELETE	/invites/:id*	-	204
acceptInvite()	PATCH	/invites/:id*	{token, password idToken}	{user, token}
validateInviteToken()	-	-	-	-
toInviteDto()	-	-	-	Invite
toCreateInviteDto()	-	-	-	Invite
Classe TokenResetPassword				

Metodo	HTTP	URI + query	Request	Response
requestReset()	POST	/auth/recovery-tokens	{email}	200 generica
verifyToken()	POST	/auth/recovery-validations	{token}	{valid}
confirmReset()	POST	/auth/recovery-confirmations	{token, password}	200

Classe Edificio

search Buildings()	GET	/buildings/	filtri + paginazione	Building[]
createBuilding()	POST	/buildings/	anagrafica edificio	Building
getBuilding()	GET	/buildings/:id*	-	Building
update Building()	PATCH	/buildings/:id*	campi edificio	Building
deleteBuilding()	DELETE	/buildings/:id*	-	204
getHistory()	GET	/buildings/:id*/readings ?startDate!, endDate!, sensorType?, interval?, limit?, sortOrder?	-	Reading[]
getRealtime()	-	-	-	-
calculate Building Efficiency()	GET	/buildings/:id*/efficiency ?startDate!, endDate!	-	{COP, ...}
buildReport Data()	GET	/reports/ ?buildingIds!, startDate!, endDate!, format?	-	PDF XLSX
toBuilding SummaryDTO()	-	-	-	Building
toBuildingDetail DTO()	-	-	-	Building
classifyHeating System()	-	-	-	-
aggregateDaily Consumption()	-	-	-	-
getCoordinates()	-	-	-	-
getAvgHist Temperature()	-	-	-	-
evaluate Efficiency Alerts()	-	-	-	-
advance Thermostat()	-	-	-	-
computeSensor Reading()	-	-	-	-

Classe TipologiaEdificio

listBuilding Types()	GET	/building-types/	-	BuildingType[]
getBuilding Type()	GET	/building-types/:id*	-	BuildingType
createBuilding Type()	POST	/building-types/	{name, description}	BuildingType
updateBuilding Type()	PATCH	/building-types/:id*	{name, description}	BuildingType

Metodo	HTTP	URI + query	Request	Response
deleteBuildingType()	DELETE	/building-types/:id*	-	204
toBuildingTypeDTO()	-	-	-	BuildingType
Classe Sensore				
listSensors()	GET	/sensors/ ?building?, sensorType?, status?	-	Sensor[]
createSensor()	POST	/sensors/	config. sensore	Sensor
getSensor()	GET	/sensors/:id*	-	Sensor
updateSensor()	PATCH	/sensors/:id*	config. + sogle	Sensor
deleteSensor()	DELETE	/sensors/:id*	-	204
getSensorHistory()	GET	/sensors/:id*/readings ?startDate?, endDate?, limit?, offset?, sortOrder?	-	Reading[]
connectAndSubscribe()	MQTT	sensors/+/readings	-	-
isActive()	-	-	-	Boolean
updateStatus()	-	-	-	-
toSensorDTO()	-	-	-	Sensor
Classe Rilevazione				
ingestReading()	MQTT	sensors/+/readings	{value, unit, timestamp}	-
listBySensor()	GET	/sensors/:id*/readings ?startDate?, endDate?, limit?, offset?, sortOrder?	-	Reading[]
listByBuilding()	GET	/buildings/:id*/readings ?startDate!, endDate!, sensorType?, interval?, limit?, sortOrder?	-	Reading[]
exportReadings()	GET	/readings/ ?buildingIds?, startDate?, endDate?, format?	-	CSV JSON
getTimeseries()	GET	/metrics/timeseries ?startDate!, endDate!, interval?	-	serie aggregate
getKpi()	GET	/metrics/	-	KPI
handleMqttMessage()	-	-	-	-
toSensorReadingDTO()	-	-	-	Reading
serializeCsv()	-	-	-	CSV
Classe Allarme				
listAlerts()	GET	/alerts/ ?building?, status?, severity?	-	Alert[]
getAlert()	GET	/alerts/:id*	-	Alert
acknowledge()	PATCH	/alerts/:id*	{status: acknowledged}	Alert
resolveManually()	PATCH	/alerts/:id*	{status: resolved}	Alert

Metodo	HTTP	URI + query	Request	Response
raiseThreshold()	-	-	-	Alert
raiseEfficiency()	-	-	-	Alert
resolveEfficiency ForBuilding()	-	-	-	-
prune ForRemoved Thresholds()	-	-	-	-
delete ForBuilding()	-	-	-	-
delete ForSensor()	-	-	-	-
compute Deviation Severity()	-	-	-	severity
toAlertDTO()	-	-	-	Alert
dispatchAlert Notifications()	-	-	-	-
Classe ConfigurazioneSistema				
getConfig()	GET	/settings/	-	Config
updateConfig()	PATCH	/settings/	{polling, notif., database}	Config
downloadBackup()	GET	/backups/	-	JSON download
getOrCreate()	-	-	-	Config
serializeConfig()	-	-	-	Config